Lecture 3

GD over real data set

- **Case study**
  - **Training Linear regression model**

Linear regression:

Linear regression:

- Input feature vector $x_i = (x_{i1}, x_{i2}, \dots, x_{id})^T \in \mathbb{R}^d$
- Model parameters (weights) $W = (w_1, \dots, w_d)^T \in \mathbb{R}^d$
- The real valued target value $y_i \in \mathbb{R}$
- Bias term $b \in \mathbb{R}$
- Hypothesis function:

$$\widehat{y_i} = f(x_i, W, b) = W^T x_i + b = w_1 x_{i1} + w_2 x_{i2} + \cdots + w_d x_{id} + b$$

$$\Rightarrow \widehat{y_i} = \sum_{j=1}^{d} w_j x_{ij} + b$$

- Error: $E_i(W, b) = (\widehat{y_i} - y_i)$

$$\Rightarrow E_i(W, b) = \left(\sum_{j=1}^{d} w_j x_{ij} + b \right) - y_i$$

- Loss function

$$\Rightarrow L(W, b) = \frac{1}{2 \times N} \sum_{i=1}^{N} E_i(W, b)^2$$

$$D = \begin{bmatrix} x_{11}, x_{12}, \dots, x_{1d}, y_1 \\ x_{21}, x_{22}, \dots, x_{2d}, y_2 \\ x_{31}, x_{32}, \dots, x_{3d}, y_3 \\ . \\ . \\ x_{N1}, x_{N2}, \dots, x_{Nd}, y_N \end{bmatrix}$$

$$X = \begin{bmatrix} x_{11}, x_{12}, \dots, x_{1d} \\ x_{21}, x_{22}, \dots, x_{2d} \\ x_{31}, x_{32}, \dots, x_{3d} \\ . \\ . \\ x_{N1}, x_{N2}, \dots, x_{Nd} \end{bmatrix} \qquad Y = \begin{bmatrix} y_1 \\ y_2 \\ y_3 \\ . \\ . \\ y_N \end{bmatrix}$$

$$W = \begin{bmatrix} w_1 \\ w_2 \\ w_3 \\ . \\ . \\ w_d \end{bmatrix} = ?$$

$$\nabla_w L(W, b) = ?$$
$$\nabla_b L(W, b) = ?$$

Linear regression (Augmented form):

Linear regression:

- Input feature vector $x_i = (x_{i1}, x_{i2}, \dots, x_{id})^T \in \mathbb{R}^d$
  - Augmented input feature vector $\tilde{x}_i = (1, x_{i1}, x_{i2}, \dots, x_{id})^T \in \mathbb{R}^{d+1}$
- Model parameters (weights) $W = (w_1, \dots, w_d)^T \in \mathbb{R}^d$
  - Augmented Model parameters (weights) $\tilde{W} = (b, w_1, \dots, w_d)^T \in \mathbb{R}^{d+1}$
- The real valued target value $y_i \in \mathbb{R}$
- **Hypothesis function:**

$$\Rightarrow f(\tilde{x}_i, \tilde{W}) = \tilde{W}^T \tilde{x}_i = 1.b + w_1 x_{i1} + w_2 x_{i2} + \cdots + w_d x_{id} = \sum_{j=1}^{d} w_j x_{ij} + b$$

- **Error:**

$$\Rightarrow E_i(\tilde{W}) = (\tilde{W}^T \tilde{x}_i) - y_i = \left(\sum_{j=1}^{d} w_j x_{ij} + b \right) - y_i$$

- **Loss function (MSE):**

$$\Rightarrow L(\tilde{W}) = \frac{1}{2 \times N} \left(\sum_{i=1}^{N} E_i(\tilde{W})^2 \right) = \frac{1}{2N} \|\tilde{X}\tilde{W} - Y\|^2$$

$$D = \begin{bmatrix} x_{11}, x_{12}, \dots, x_{1d}, y_1 \\ x_{21}, x_{22}, \dots, x_{2d}, y_2 \\ x_{31}, x_{32}, \dots, x_{3d}, y_3 \\ \vdots \\ x_{N1}, x_{N2}, \dots, x_{Nd}, y_N \end{bmatrix}$$

$$\tilde{X} = \begin{bmatrix} 1, x_{11}, x_{12}, \dots, x_{1d} \\ 1, x_{21}, x_{22}, \dots, x_{2d} \\ 1, x_{31}, x_{32}, \dots, x_{3d} \\ \vdots \\ 1, x_{N1}, x_{N2}, \dots, x_{Nd} \end{bmatrix} \qquad Y = \begin{bmatrix} y_1 \\ y_2 \\ y_3 \\ \vdots \\ y_N \end{bmatrix}$$

$$\tilde{W} = \begin{bmatrix} b \\ w_1 \\ w_2 \\ w_3 \\ \vdots \\ w_d \end{bmatrix} = ?$$

$$\nabla_{\tilde{W}} L(\tilde{W}) = ?$$

Direct gradient calculation

- $L(\widetilde{W}) = \frac{1}{2\times N}\left(\sum_{i=1}^{N} E_i(\widetilde{W})^2\right) = \frac{1}{2N}\left\|\tilde{X}\widetilde{W} - Y\right\|^2 = \frac{1}{2N}\left(\tilde{X}\widetilde{W} - Y\right)^T\left(\tilde{X}\widetilde{W} - Y\right)$ %Since $\|\boldsymbol{a}\|_2^2 = \boldsymbol{a}^T\boldsymbol{a}$
- **Since $(\boldsymbol{A}-\boldsymbol{B})^T(\boldsymbol{A}-\boldsymbol{B}) = (\boldsymbol{A}^T-\boldsymbol{B}^T)(\boldsymbol{A}-\boldsymbol{B})$** **% Since $(\boldsymbol{A}-\boldsymbol{B})^T = \boldsymbol{A}^T-\boldsymbol{B}^T$**
  $= \boldsymbol{A}^T\boldsymbol{A} - \boldsymbol{A}^T\boldsymbol{B} - \boldsymbol{B}^T\boldsymbol{A} + \boldsymbol{B}^T\boldsymbol{B}$ % Valid for both vectors and matrices, provided A and B have the same dimensions.
- ***Here $\boldsymbol{A} = \tilde{X}\widetilde{W}$ and $\boldsymbol{B} = \boldsymbol{Y}$***
  - $L(\widetilde{W}) = \frac{1}{2N}\left(\tilde{X}\widetilde{W} - Y\right)^T\left(\tilde{X}\widetilde{W} - Y\right) = \frac{1}{2N}\left[(\widetilde{X}\widetilde{W})^T(\tilde{X}\widetilde{W}) - (\widetilde{X}\widetilde{W})^T\boldsymbol{Y} - \boldsymbol{Y}^T(\tilde{X}\widetilde{W}) + \boldsymbol{Y}^T\boldsymbol{Y}\right]$
  - $L(\widetilde{W}) = \frac{1}{2N}\left[\widetilde{W}^T\tilde{X}^T\tilde{X}\widetilde{W} - \widetilde{W}^T\tilde{X}^T\boldsymbol{Y} - \boldsymbol{Y}^T\tilde{X}\widetilde{W} + \boldsymbol{Y}^T\boldsymbol{Y}\right]$ **% Since $(\widetilde{X}\widetilde{W})^T = \widetilde{W}^T\tilde{X}^T$**
  - $L(\widetilde{W}) = \frac{1}{2N}\left[\widetilde{W}^T\tilde{X}^T\tilde{X}\widetilde{W} - 2\widetilde{W}^T\tilde{X}^T\boldsymbol{Y} + \boldsymbol{Y}^T\boldsymbol{Y}\right]$ *% Since* $\boldsymbol{Y}^T\tilde{X}\widetilde{W} = \widetilde{W}^T\tilde{X}^T\boldsymbol{Y}$
- We have a standard formula: $\nabla_W(W^TAW) = (A + A^T)W$
- $\nabla_{\widetilde{W}}L(\widetilde{W}) = \frac{1}{2N}\left[\nabla_{\widetilde{W}}\left(\widetilde{W}^T\tilde{X}^T\tilde{X}\widetilde{W}\right) - 2\nabla_{\widetilde{W}}\left(\widetilde{W}^T\tilde{X}^T\boldsymbol{Y}\right) + \nabla_{\widetilde{W}}(Y^TY)\right]$
- $\nabla_{\widetilde{W}}L(\widetilde{W}) = \frac{1}{2N}\left[\left(\left(\tilde{X}^T\tilde{X} + (\widetilde{X}^T\tilde{X})^T\right)\widetilde{W} - 2\widetilde{X}^TY + 0\right]\right.$
- $\nabla_{\widetilde{W}}L(\widetilde{W}) = \frac{1}{2N}\left[\,2\tilde{X}^T\tilde{X}\widetilde{W} - 2\widetilde{X}^TY\right]$ % Since $(\widetilde{X}^T\tilde{X})^T = \tilde{X}^T\tilde{X}$

$$\boldsymbol{\nabla_{\widetilde{W}}L(\widetilde{W}) = \frac{1}{N}\left(X^TX\widetilde{W} - X^TY\right) = \frac{1}{N}X^T\left(X\widetilde{W} - Y\right)}$$

- For a stationary point:

$$\boldsymbol{\nabla_{\widetilde{W}}L(\widetilde{W}) = 0 \Rightarrow \frac{1}{N}\left(X^TX\widetilde{W} - X^TY\right) = 0 \Rightarrow \widetilde{W} = \left(X^TX\right)^{-1}X^TY}$$ **(Normal Equation or closed form solution)**

Normal Equation	**Gradient Descent**
Closed-form/linear-system solution	Iterative optimization
No learning rate required	Requires learning rate
No epochs required	Requires iterations/epochs
Very effective for small/moderate feature dimensions	Better suited to large-scale problems
Can suffer from singular/ill-conditioned matrices	Does not require matrix inversion
Computational cost grows strongly with feature dimension	Can use mini-batches/SGD
Mainly for linear least-squares problems	Can handle much more general differentiable models

Calculation of $\nabla_w L(W)$ for linear regression

- For the sake of simplicity, let $d = 2, N = 2$.
- $X = \begin{bmatrix} x_{11} & x_{12} \\ x_{21} & x_{22} \end{bmatrix}, W = \begin{bmatrix} w_1 \\ w_2 \end{bmatrix}, Y = \begin{bmatrix} y_1 \\ y_2 \end{bmatrix}$, Bias term b

- Hypothesis function: $f(x_i, W, b) = W^T x_i = w_1 x_{i1} + w_2 x_{i2} + b$
- Error: $E_i(W, b) = (w_1 x_{i1} + w_2 x_{i2} + b) - y_i$
- Loss function:
  - $L(W, b) = \frac{1}{2\times 2} \sum_{i=1}^{2} E_i(W, b)^2 = \frac{1}{2} \sum_{i=1}^{2} \left((w_1 x_{i1} + w_2 x_{i2} + b) - y_i\right)^2$
  - $L(W, b) = \frac{1}{2\times 2} [(w_1 x_{11} + w_2 x_{12} + b) - y_1]^2 + \frac{1}{2\times 2} [(w_1 x_{21} + w_2 x_{22} + b) - y_2]^2$

- Now, gradient calculation:
- $\frac{\partial L}{\partial w_1} = \frac{1}{2\times 2} . 2[(w_1 x_{11} + w_2 x_{12} + b) - y_1] \times x_{11} + \frac{1}{2\times 2} . 2[(w_1 x_{21} + w_2 x_{22} + b) - y_2] \times x_{21}$
- $\frac{\partial L}{\partial w_2} = \frac{1}{2\times 2} . 2[(w_1 x_{11} + w_2 x_{12} + b) - y_1] \times x_{12} + \frac{1}{2\times 2} . 2[(w_1 x_{21} + w_2 x_{22} + b) - y_2] \times x_{22}$
- $\frac{\partial L}{\partial b} = \frac{1}{2} . [(w_1 x_{11} + w_2 x_{12} + b) - y_1] \times 1 + \frac{1}{2} . [(w_1 x_{21} + w_2 x_{22} + b) - y_2] \times 1$

- $\nabla_W L(W, b) = \begin{bmatrix} \frac{\partial L}{\partial w_1} \\ \frac{\partial L}{\partial w_2} \end{bmatrix} = \frac{1}{2} \begin{bmatrix} x_{11} & x_{21} \\ x_{12} & x_{22} \end{bmatrix} \begin{bmatrix} (w_1 x_{11} + w_2 x_{12} + b) - y_1 \\ (w_1 x_{21} + w_2 x_{22} + b) - y_2 \end{bmatrix} = \frac{1}{2} X^T (XW + b - Y)$
- $\nabla_b L(W, b) = \frac{\partial L}{\partial b} = \frac{1}{2} 1^T (XW + b - Y)$

Example (gradient calculation for linear regression)

- For $X \in \mathbb{R}^{2\times 2}, W \in \mathbb{R}^2, Y \in \mathbb{R}^2$
- $\nabla_W L(W,b) = \frac{1}{2} X^T (XW + b - Y)$
- $\nabla_b L(W,b) = \frac{\partial L}{\partial b} = \frac{1}{2} 1^T (XW + b - Y)$

- General linear regression gradient calculation,
- For $X \in \mathbb{R}^{N\times d}, W \in \mathbb{R}^d, Y \in \mathbb{R}^N$
- $\nabla_W L(W,b) = \frac{1}{N} X^T (XW + b - Y) = \frac{1}{N} X^T E$
- $\nabla_b L(W,b) = \frac{\partial L}{\partial b} = \frac{1}{N} 1^T (XW + b - Y) = \frac{1}{N} 1^T E$

- Given
- Feature matrix $X = \begin{bmatrix} 1 & 2 \\ 3 & 4 \end{bmatrix}$, Weight vector $W = \begin{bmatrix} 0.5 \\ 0.5 \end{bmatrix}$, Bias term $\mathrm{b} = 0.5$, Target vector $Y = \begin{bmatrix} 1 \\ 2 \end{bmatrix}$,
- Define the **least squares loss function** $L(W,b)$ for **linear regression**.
- Derive the gradient of $L(W,\mathrm{b})$ with respect to the weight W and bias b.

- Hypothesis function of linear regression: $f(x_i, W, b) = W^T x_i = w_1 x_{i1} + w_2 x_{i2} + b$
- Error: $E_i(W,b) = (w_1 x_{i1} + w_2 x_{i2} + b) - y_i$
- Loss function:
  - $L(W,b) = \frac{1}{2\times 2} \sum_{i=1}^{2} E_i(W,b)^2 = \frac{1}{2} \sum_{i=1}^{2} \left((w_1 x_{i1} + w_2 x_{i2} + b) - y_i\right)^2$
  - $L(W,b) = \frac{1}{2\times 2} [(w_1 x_{11} + w_2 x_{12} + b) - y_1]^2 + \frac{1}{2\times 2} [(w_1 x_{21} + w_2 x_{22} + b) - y_2]^2$

Example (contd.)

- Since
  - $\nabla_W L(W,b) = \frac{1}{N} X^T (XW + b - Y)$
  - $\nabla_b L(W,b) = \frac{\partial L}{\partial b} = \frac{1}{N} 1^T (XW + b - Y)$
- Prediction by linear regression:
  - $XW + b = \begin{bmatrix} 1 & 2 \\ 3 & 4 \end{bmatrix} \begin{bmatrix} 0.5 \\ 0.5 \end{bmatrix} + 0.5 = \begin{bmatrix} 2 \\ 4 \end{bmatrix}$
- Error calculation:
  - $(XW + b) - Y = \begin{bmatrix} 2 \\ 4 \end{bmatrix} - \begin{bmatrix} 1 \\ 2 \end{bmatrix} = \begin{bmatrix} 1 \\ 2 \end{bmatrix}$
- Loss function:
  - $L(W,b) = \frac{1}{2N} \|(XW + b) - Y\|^2$
- Compute
  - $\nabla_W L(W,b) = \frac{1}{2} X^T (XW + b - Y)$:
    - $\begin{bmatrix} 1 & 3 \\ 2 & 4 \end{bmatrix} \begin{bmatrix} 1 \\ 2 \end{bmatrix} = \frac{1}{2} * \begin{bmatrix} 7 \\ 10 \end{bmatrix} = \begin{bmatrix} 3.5 \\ 5 \end{bmatrix}$
- $\nabla_b L(W,b) = \frac{\partial L}{\partial b} = \frac{1}{N} 1^T (XW + b - Y) = \frac{1}{2} * [1,1] \begin{bmatrix} 1 \\ 2 \end{bmatrix} = 1.5$

Example (contd.)

- Given
- Feature matrix $X = \begin{bmatrix} 1 & 2 \\ 3 & 4 \end{bmatrix}$, Weight vector $W = \begin{bmatrix} 0.5 \\ 0.5 \end{bmatrix}$, Bias term $\mathrm{b} = 0.5$, Target vector $Y = \begin{bmatrix} 1 \\ 2 \end{bmatrix}$,
- Define the **least squares loss function** $L(W, b)$ for **linear regression**.
- Derive the gradient of $L(W,\mathrm{b})$ with respect to the weight W and bias b.

- **Augmented form**:
- Feature matrix $\tilde{X} = \begin{bmatrix} 1 & 1 & 2 \\ 1 & 3 & 4 \end{bmatrix}$, Weight vector $\widetilde{W} = \begin{bmatrix} 0.5 \\ 0.5 \\ 0.5 \end{bmatrix}$, Target vector $Y = \begin{bmatrix} 1 \\ 2 \end{bmatrix}$
- Compute prediction by linear regression model:
  - $\hat{Y} = \tilde{X}\widetilde{W} = \begin{bmatrix} 1 & 1 & 2 \\ 1 & 3 & 4 \end{bmatrix} \begin{bmatrix} 0.5 \\ 0.5 \\ 0.5 \end{bmatrix} = \begin{bmatrix} 2 \\ 4 \end{bmatrix}$
- Error calculation:
  - $E = (\hat{Y} - Y) = \begin{bmatrix} 2 \\ 4 \end{bmatrix} - \begin{bmatrix} 1 \\ 2 \end{bmatrix} = \begin{bmatrix} 1 \\ 2 \end{bmatrix}$
- Loss function $L(\widetilde{W}) = \frac{1}{2N} \|E\|^2 = \frac{1}{2N} \left\| (\tilde{X}\widetilde{W}) - Y \right\|^2$
- Compute gradient
  - $\nabla_{\widetilde{W}} L(\widetilde{W}) = \frac{1}{2} \tilde{X}^T (\tilde{X}\widetilde{W} - Y) = \frac{1}{2} \begin{bmatrix} 1 & 1 \\ 1 & 3 \\ 2 & 4 \end{bmatrix} \begin{bmatrix} 1 \\ 2 \end{bmatrix} = \frac{1}{2} \begin{bmatrix} 3 \\ 7 \\ 10 \end{bmatrix} = \begin{bmatrix} 1.5 \\ 3.5 \\ 5 \end{bmatrix}$

- General linear regression gradient calculation,
- For $X \in \mathbb{R}^{N \times d}, W \in \mathbb{R}^d, Y \in \mathbb{R}^N$
- $\nabla_W L(W, b) = \frac{1}{N} X^T (XW + b - Y)$
- $\nabla_b L(W, b) = \frac{\partial L}{\partial b} = \frac{1}{N} 1^T (XW + b - Y)$

GD for training linear regression model

GD (Vanilla GD) for training linear regression model:

%Inputs

X: Feature matrix of size $N \times d$ ($X \in \mathbb{R}^{N \times d}$), $\tilde{X}$: Augmented matrix of size $N \times (d+1)$($\tilde{X} \in \mathbb{R}^{N \times (d+1)}$),

Y: Target vector of size $N \times 1$ ($Y \in \mathbb{R}^{N \times 1}$)

$\widetilde{W}^{(1)} \in \mathbb{R}^{(d+1) \times 1}$ Initial model parameter **with bias term**

α: learning rate

max _it: Maximum number of iterations

$k = 1$ % Initialize iteration counter

%Output

W_{final}: Final optimized weight vector $\widetilde{W}^{(k)} \in \mathbb{R}^{(d+1) \times 1}$ after k iterations

Begin

While $k <$ max _it **do**

$\hat{Y}^{(k)} = X^T * \widetilde{W}^{(k)}$; % Compute predictions based on current weights (Hypothesis function of **linear regression**)

$E^{(k)} = (\hat{Y}^{(k)} - Y)$ % Difference between predicted $\hat{Y}^{(k)} \in \mathbb{R}^{N \times 1}$ and true values $Y \in \mathbb{R}^{N \times 1}$

$L(\widetilde{W}^{(k)}) = \frac{1}{2N} \left\| E^{(k)} \right\|^2$ %Least square loss function (just for monitoring, not the part of update)

$\nabla_{\widetilde{W}} L(\widetilde{W}^{(k)}) = \frac{2}{2N} * X^T * E^{(k)}$ %Gradient of loss function with respect to weight vector (model parameters), depends on whole X^T

$\widetilde{W}^{(k+1)} = \widetilde{W}^{(k)} - \alpha \nabla_{\widetilde{W}} L(\widetilde{W}^{(k)})$ % Update model parameters using calculated gradient and learning late

$k = k + 1$; % Increment k

EndWhile

End

GD limitations:

- **Computational Cost per Iteration**:
- GD requires the computation of the gradient over the entire dataset in each iteration, which can be computationally expensive, especially for large datasets.

- **Memory Requirements:**
- GD needs to store the entire dataset in memory to compute the gradient, which can be prohibitive for large datasets.

- **Scalability issue:**
- GD does not scale well with very large datasets because of the need to process the entire dataset at each step

- **Convergence Speed:**
- GD can be slow to converge, particularly in large datasets or high-dimensional spaces, because it must compute the gradient for the entire dataset at each step.

- **Stagnation issue:**
- **GD** follows a deterministic path, which may cause it to get stuck in local minima, particularly in non-convex optimization problems.

- **Online Streaming Context:**
- In online streaming scenarios, data is not available in a static, complete form. Instead, it arrives continuously, often in small batches or individual data points. Traditional GD cannot operate incrementally; it needs the entire dataset to calculate the gradient at each iteration

SGD for linear regression :

SGD for training linear regression model:

%Inputs

X: Feature matrix of size $N \times d$ ($X \in \mathbb{R}^{N \times d}$), $\tilde{X}$: Augmented matrix of size $N \times (d+1)$($\tilde{X} \in \mathbb{R}^{N \times (d+1)}$),

Y: Target vector of size $N \times 1$ ($Y \in \mathbb{R}^{N \times 1}$)

$\widetilde{W}^{(1)} \in \mathbb{R}^{(d+1)\times 1}$ Initial model parameter **with biased term**

α: learning rate

max _it: Maximum number of iterations

$k = 1$ % Initialize iteration counter

%Output

W_{final} :Final optimized weight vector $\widetilde{W}^{(k)} \in \mathbb{R}^{(d+1)\times 1}$ after k iterations

Begin

While $k <$ max _it **do**

$\langle \tilde{x}_i, y_i \rangle \leftarrow$ DrawRandomDataPoint $D = [X, Y]$ %Draw a random example with replacement (Same example can be selected multiple times)

$\hat{y}_i = \tilde{x}_i^T * \widetilde{W}^{(k)}$% Compute predictions for the selected data point (Prediction using current weight by Hypothesis function)

$E_i = (\hat{y}_i - y_i)$ % Difference between predicted value $\hat{y}_i$ and true value y_i of the randomly selected data point x_i

$L(\widetilde{W}^{(k)}) = \frac{1}{2} E_i^2$ % Least square loss function for linear regression with respect to x_i (just for monitoring, not the part of update)

$\nabla_{\widetilde{W}} L(\widetilde{W}^{(k)}) = \tilde{x}_i * E_i$ % Gradient of loss function with respect to weight vector (model parameters), depends on single x_i

$\widetilde{W}^{(k+1)} = \widetilde{W}^{(k)} - \alpha \nabla_{\widetilde{W}} L(\widetilde{W}^{(k)})$% Update model parameters using calculated gradient and learning

$k = k + 1$; % Increment k

EndWhile

End

For a single data point $(\tilde{x}_i, y_i)$:

- The prediction is $\hat{y}_i = \tilde{x}_i^T * \widetilde{W}$
- Error $E_i = (\hat{y}_i - y_i)$
- LSE loss function $L(W) = \frac{1}{2}(E_i)^2 = \frac{1}{2}\left(\tilde{x}_i^T * W - y_i\right)^2$
- Gradient $\nabla_W L(W) = \left(\tilde{x}_i^T * W - y_i\right)(\tilde{x}_i) = \tilde{x}_i * E_i$

Mini batch SGD for training linear regression model:

Mini batch SGD for training linear regression model:

%Inputs

X: Feature matrix of size $N \times d$ ($X \in \mathbb{R}^{N \times d}$), $\tilde{X}$: Augmented matrix of size $N \times (d+1)$($\tilde{X} \in \mathbb{R}^{N \times (d+1)}$),

Y: Target vector of size $N \times 1$ ($Y \in \mathbb{R}^{N \times 1}$)

$\widetilde{W}^{(1)} \in \mathbb{R}^{(d+1) \times 1}$ Initial model parameter **with bias term**

α: learning rate

max_it: Maximum number of iterations

$k = 1$ % Initialize iteration counter

Minibatch size: Size of each mini-batch (integer value)

%Output

W_{final} :Final optimized weight vector $W^{(k)} \in \mathbb{R}^{(d+1) \times 1}$ after k iterations

Begin

While $k <$ max_it **do**

$\langle \tilde{X}_{mini}, Y_{mini} \rangle \leftarrow$ DrawRandomMiniBatch(X, Y, Minibatch size) %Draw a random mini-batch with replacement

$\hat{Y}_{mini} = \tilde{X}_{mini}^{T} * \widetilde{W}^{(k)}$ % Compute predictions for selected mini-batch (Prediction using current weight by Hypothesis function)

$E_{mini} = (\hat{Y}_{mini} - Y_{mini})$ % Difference between predicted values $\hat{Y}_{mini}$ and true values Y_{mini} of the selected mini-batch

$L(\widetilde{W}^{(k)}) = \frac{1}{2 \times \text{Minibatch size}} \|E_{mini}\|^2$ % LSE Loss function for the mini-batch

$\nabla_{\widetilde{W}^{(k)}} L(\widetilde{W}^{(k)}) = \frac{1}{\text{Minibatch size}} \tilde{X}_{mini}^{T} * E_{mini}$ % Gradient with respect to weights for the mini-batch

$\widetilde{W}^{(k+1)} = \widetilde{W}^{(k)} - \alpha \nabla_{\widetilde{W}} L(\widetilde{W}^{(k)})$ % Update the weights using the calculated gradient and learning rate

$k = k + 1$; % Increment k

EndWhile

End

Example:

- Given the following data for the simple linear regression problem:
- Feature matrix $X = \begin{bmatrix} 1 & 2 \\ 3 & 4 \end{bmatrix}$, Weight vector $W = \begin{bmatrix} 0.5 \\ 0.5 \end{bmatrix}$, Bias term b $= 1$, Target vector $Y = \begin{bmatrix} 5 \\ 7 \end{bmatrix}$,
- Learning rate $\alpha = 0.01$,
- Perform 2 iterations of GD and SGD to update the weights W and bias b. Show all intermediate steps, including predictions, errors, gradient calculations, Loss function values and weight updates for each iteration.

- Augmented form of the asked question:
  - $\tilde{X} = \begin{bmatrix} 1 & 1 & 2 \\ 1 & 3 & 4 \end{bmatrix}, \widetilde{W}^{(1)} = \begin{bmatrix} 1 \\ 0.5 \\ 0.5 \end{bmatrix}, Y = \begin{bmatrix} 5 \\ 7 \end{bmatrix}, \alpha = 0.01$
- **GD:**
- **Iteration 1:**
- Prediction by hypothesis function of linear regression:
  - $\tilde{Y}^{(1)} = \tilde{X}\widetilde{W}^{(1)} = \begin{bmatrix} 1 & 1 & 2 \\ 1 & 3 & 4 \end{bmatrix} \begin{bmatrix} 1 \\ 0.5 \\ 0.5 \end{bmatrix} = \begin{bmatrix} 2.5 \\ 4.5 \end{bmatrix}$
- Compute error $E^{(1)} = \tilde{Y}^{(1)} - Y = \begin{bmatrix} 2.5 \\ 4.5 \end{bmatrix} - \begin{bmatrix} 5 \\ 7 \end{bmatrix} = \begin{bmatrix} -2.5 \\ -2.5 \end{bmatrix}$
- Loss function
- $L\left(\widetilde{W}^{(1)}\right) = \frac{1}{2.2}\left\|E^{(1)}\right\|^2 = \frac{1}{4}[(-2.5)^2 + (-2.5)^2] = 3.125$

- Compute gradient
  - $\nabla_{\widetilde{W}} L(\widetilde{W}^{(1)}) = \frac{1}{2}\tilde{X}^T E^{(1)} = \frac{1}{2}\begin{bmatrix} 1 & 1 & 2 \\ 1 & 3 & 4 \end{bmatrix}^T \begin{bmatrix} -2.5 \\ -2.5 \end{bmatrix}$
    $= \frac{1}{2}\begin{bmatrix} 1 & 1 \\ 1 & 3 \\ 2 & 4 \end{bmatrix} \begin{bmatrix} -2.5 \\ -2.5 \end{bmatrix} = \begin{bmatrix} -2.5 \\ -4.5 \\ -7.5 \end{bmatrix}$
- Update the weights
  - $\widetilde{W}^{(2)} = \widetilde{W}^{(1)} - \alpha \nabla_{\widetilde{W}} L\left(\widetilde{W}\right)^{(1)}$
  - $\widetilde{W}^{(2)} = \begin{bmatrix} 1 \\ 0.5 \\ 0.5 \end{bmatrix} - 0.01 \begin{bmatrix} -2.5 \\ -4.5 \\ -7.5 \end{bmatrix} = \begin{bmatrix} 1.025 \\ 0.545 \\ 0.575 \end{bmatrix}$

Example (contd.)

- **Iteration 2:**
- Prediction by hypothesis function of linear regression:
  - $\tilde{Y}^{(2)} = \tilde{X}\widetilde{W}^{(2)} = \begin{bmatrix}1 & 1 & 2\\ 1 & 3 & 4\end{bmatrix}\begin{bmatrix}1.025\\ 0.545\\ 0.575\end{bmatrix} = \begin{bmatrix}2.72\\ 4.96\end{bmatrix}$
- Compute error $E^{(2)} = \tilde{Y}^{(2)} - Y = \begin{bmatrix}2.72\\ 4.96\end{bmatrix} - \begin{bmatrix}5\\ 7\end{bmatrix} = \begin{bmatrix}-2.28\\ -2.04\end{bmatrix}$
- Loss function
- $L\left(\widetilde{W}^{(2)}\right) = \frac{1}{2.2}\left\|E^{(2)}\right\|^2 = \frac{1}{4}[(-2.28)^2 + (-2.04)^2] = 2.34$

- Compute gradient
  - $\nabla_{\widetilde{W}} L(\widetilde{W}^{(2)}) = \frac{1}{2}\tilde{X}^T E^{(2)} = \frac{1}{2}\begin{bmatrix}1 & 1 & 2\\ 1 & 3 & 4\end{bmatrix}^T \begin{bmatrix}-2.28\\ -2.04\end{bmatrix}$
  $= \frac{1}{2}\begin{bmatrix}1 & 1\\ 1 & 3\\ 2 & 4\end{bmatrix}\begin{bmatrix}-2.28\\ -2.04\end{bmatrix} = \begin{bmatrix}-2.16\\ -4.2\\ -6.36\end{bmatrix}$
- Update the weights
  - $\widetilde{W}^{(3)} = \widetilde{W}^{(2)} - \alpha\nabla_{\widetilde{W}} L(\widetilde{W}^{(2)})$
  - $\widetilde{W}^{(3)} = \begin{bmatrix}1.025\\ 0.545\\ 0.575\end{bmatrix} - 0.01\begin{bmatrix}-2.16\\ -4.2\\ -6.36\end{bmatrix} = \begin{bmatrix}1.0466\\ 0.587\\ 0.6386\end{bmatrix}$

Example (contd.)

- Augmented form of the asked question:
  - $\tilde{X} = \begin{bmatrix} 1 & 1 & 2 \\ 1 & 3 & 4 \end{bmatrix}, \widetilde{W}^{(1)} = \begin{bmatrix} 1 \\ 0.5 \\ 0.5 \end{bmatrix}, Y = \begin{bmatrix} 5 \\ 7 \end{bmatrix}, \alpha = 0.01$
- **SGD:**
- **Iteration 1:**
- Randomly, if the first data point from $\tilde{X}$ is selected
- $\widetilde{x_1} = \begin{bmatrix} 1 \\ 1 \\ 2 \end{bmatrix}$ with $y_1 = 5$
- Prediction by hypothesis function of linear regression:
  - $\widetilde{y_1}^{(1)} = \widetilde{x_1}^T \widetilde{W}^{(1)} = [1 \quad 1 \quad 2] \begin{bmatrix} 1 \\ 0.5 \\ 0.5 \end{bmatrix} = 2.5$
- Compute error $E^{(1)} = \widetilde{y_1}^{(1)} - y_1 = (2.5 - 5) = -2.5$
- Loss function
- $L\left(\widetilde{W}^{(1)}\right) = \frac{1}{2.2}\left(E^{(1)}\right)^2 = \frac{1}{4} 6.25 = 1.5625$

- Compute gradient
  - $\nabla_{\widetilde{W}} L(\widetilde{W}^{(1)}) = \frac{1}{2} \widetilde{x_1} E^{(1)} = \frac{1}{2} \begin{bmatrix} 1 \\ 1 \\ 2 \end{bmatrix} * (-2.5) = \begin{bmatrix} -1.25 \\ -1.25 \\ -2.5 \end{bmatrix}$
- Update the weights
  - $\widetilde{W}^{(2)} = \widetilde{W}^{(1)} - \alpha \nabla_{\widetilde{W}} L\left(\widetilde{W}\right)^{(1)}$
  - $\widetilde{W}^{(2)} = \begin{bmatrix} 1 \\ 0.5 \\ 0.5 \end{bmatrix} - 0.01 \begin{bmatrix} -1.25 \\ -1.25 \\ -2.5 \end{bmatrix} = \begin{bmatrix} 0.9875 \\ 0.375 \\ 0.475 \end{bmatrix}$

Example (contd.)

- **Iteration 2:**
- Randomly, if the second data point from $\tilde{X}$ is selected
- $\widetilde{x_2} = \begin{bmatrix} 1 \\ 3 \\ 4 \end{bmatrix}$ with $y_2 = 7$
- Prediction by hypothesis function of linear regression:
  - $\widetilde{y_2}^{(2)} = \widetilde{x_2}^T \widetilde{W}^{(2)} = [1 \quad 3 \quad 4] \begin{bmatrix} 0.9875 \\ 0.375 \\ 0.475 \end{bmatrix} = 4.0125$
  - Compute error $E^{(2)} = \widetilde{y_2}^{(2)} - y_2 = (4.0125 - 7) = -2.9875$
- Loss function
- $L(\widetilde{W}^{(2)}) = \frac{1}{2.2}(E^{(1)})^2 = \frac{1}{4} 8.925156 = 2.2312$

- Compute gradient
  - $\nabla_{\widetilde{W}} L(\widetilde{W}^{(2)}) = \frac{1}{2} \widetilde{x_2} E^{(2)} = ?$
- Update the weights
  - $\widetilde{W}^{(3)} = \widetilde{W}^{(2)} - \alpha \nabla_{\widetilde{W}} L(\widetilde{W})^{(2)}$
  - $\widetilde{W}^{(3)} = ?$